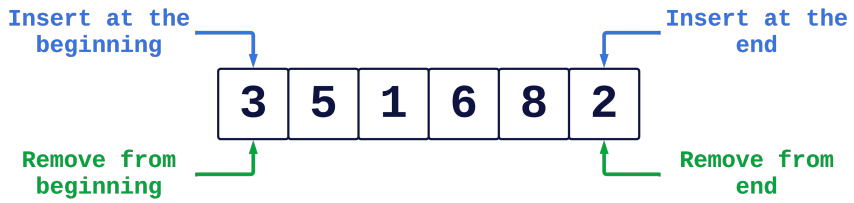
Learning Objectives

Learners will be able to…

- **Define a deque**
- **Differentiate a deque from stacks, queues, and priority queues**
- **Implement a deque**

Deque

A **deque** (pronounced as 'deck'), stands for **double-ended queue**. Unlike a regular queue, where elements can be added to the back and removed from the front, a deque allows adding and removing elements from both ends. Deques are a linear data structure that provide the functionality of both a stack and a queue. Deques can be implemented using various underlying data structures such as arrays, linked lists, or even a hybrid approach.



Core Operations

Deques provide the following key operations:

- **addFront(element)**: Adds an element to the front of the deque.
- **addRear(element)**: Adds an element to the rear of the deque.
- **removeFront()**: Removes the front element from the deque.
- **removeRear()**: Removes the rear element from the deque.
- **peekFront()**: Returns the front element without removing it.
- **peekRear()**: Returns the rear element without removing it.
- **isEmpty()**: Checks if the deque is empty.
- **size()**: Returns the number of elements in the deque.

When to Use a Deque?

Deques are particularly useful in scenarios where you need dynamic and flexible data storage with efficient operations at both ends. For example, they can be used in certain algorithms, like sliding window problems, palindrome checking, etc.

We will delve deeper into the various implementations of deques. We will also cover the time complexities associated with these operations in different types of implementations, namely:

- Array-based deques
- Linked List-based deques

Array-based Implementation

Representing a Deque with an Array

We will explore the array-based implementation of a deque. The array-based approach is straightforward and offers constant-time access to elements. However, one of the limitations is that the array size is fixed, meaning you'll have to resize the array if the deque grows beyond its initial capacity.

This implementation of a deque will be circular in nature. That means, if you increment a pointer in the last element or decrement a pointer at index 0, the pointer will "wrap back around" to the other end of the array. This is why you will use modulo (`%`) when calculating new values for `front` and `rear`.

Key Operations and Their Time Complexities

The array-based implementation of deque is efficient for small datasets but may require resizing for larger datasets. Here are the key operations for an array-based deque and their respective time complexities:

- **addFront(element)**: $O(1)$ or $O(n)$ if resizing is needed
- **addRear(element)**: $O(1)$ or $O(n)$ if resizing is needed
- **removeFront()**: $O(1)$
- **removeRear()**: $O(1)$
- **peekFront()**: $O(1)$
- **peekRear()**: $O(1)$
- **isEmpty()**: $O(1)$
- **isFull()**: $O(1)$
- **size()**: $O(1)$

Implementation

Start by creating the `ArrayDeque` class. It has four attributes: `capacity` which is the size of the array, `size` which represents the number of elements in the deque, `front` which represents the front of the deque, `rear` which represents the rear of the deque, and `dequeArray` which is the actual array used to store elements. In the constructor, pass it the size of the array. Set `capacity` to the parameter, instantiate `dequeArray` with the given size, set `front` to `0`, and set `rear` to `-1`.

```
class ArrayDeque {
private:
    int capacity;
    int size;
    int front;
    int rear;
    int* dequeArray;

public:
    // Constructor
    ArrayDeque(int c) {
        capacity = c;
        size = 0;
        front = 0;
        rear = -1;
        dequeArray = new int[capacity];
    }

    // Destructor
    ~ArrayDeque() {
        delete[] dequeArray;
    }
};
```

When adding an element to the front of the deque, start by checking to see if it is full. If so, throw an exception with a message for the user. Since we want the deque to be circular, we need to subtract `1` from `front` and add `capacity`. Then take this result and perform modulo with `capacity`. Set the element at index `front` to the given `value` and increment `size` by `1`.

```
    // Add element to the front
    void addFront(int value) {
        if (isFull()) {
            throw std::overflow_error("Deque is full");
        }

        front = (front - 1 + capacity) % capacity;
        dequeArray[front] = value;
        size++;
    }
```

When adding an element to the rear of the deque, check to see if the deque is full. If so, throw an exception with a message for the user. Calculate a new value for `rear` by adding `1` and taking the modulo with `capacity`. Set the element at index `rear` to the given `value` and increment `size` by `1`.

```
    // Add element to the rear
    void addRear(int value) {
        if (isFull()) {
            throw std::overflow_error("Deque is full");
        }

        rear = (rear + 1) % capacity;
        dequeArray[rear] = value;
        size++;
    }
```

To remove an element from the front of the deque, first check to see if it is empty. If so, throw an exception with a message for the user. Then create a temporary variable with the value of the element at index `front`. We are going to update `front` by adding `1` and taking the modulo with `capacity`. Decrement `size` by `1` and return the temporary variable.

```
    // Remove element from the front
    int removeFront() {
        if (isEmpty()) {
            throw std::underflow_error("Deque is empty");
        }

        int temp = dequeArray[front];
        front = (front + 1) % capacity;
        size--;
        return temp;
    }
```

The `removeRear` method will remove an element from the rear of a deque. First, check to see if the deque is empty. If so, throw an error with a message for the user. Then create a temporary variable that stores the value at index `rear`. Subtract `1` from `rear` and take the modulo with `capacity`. Decrement `size` by `1` and return the temporary variable.

```
    // Remove element from the rear
    int removeRear() {
        if (isEmpty()) {
            throw std::underflow_error("Deque is empty");
        }

        int temp = dequeArray[rear];
        rear = (rear - 1) % capacity;
        size--;
        return temp;
    }
```

To peek at either front or rear of the deque, first check to see if it is empty. If so, throw an exception with a message for the user. Then return the element at index `front` (peek at the front) or index `rear` (peek at the rear). Do not increment or decrement the pointers.

```
    // Peek at the front element
    int peekFront() {
        if (isEmpty()) {
            throw std::underflow_error("Deque is empty");
        }

        return dequeArray[front];
    }

    // Peek at the rear element
    int peekRear() {
        if (isEmpty()) {
            throw std::underflow_error("Deque is empty");
        }

        return dequeArray[rear];
    }
```

The `ArrayDeque` class needs a few helper methods. The two most commonly used are `isEmpty` and `isFull`. Both of these methods return a boolean value. To check if the deque is empty, ask if its size (not capacity) is `0`. A deque is full when the size of the data structure is equal to the capacity of the array.

```
    // Check if the deque is empty
    bool isEmpty() {
        return (size == 0);
    }

    bool isFull() {
        return (size == capacity);
    }
```

The last helper method is an accessor method that returns the value of the `size` attribute.

```
    int getSize() {
        return size;
    }
```

Create the `main` method which instantiates an array-based `deque` with a capacity of `5`. Add the elements `1` and `2`. Then peek at the front element. Add elements `3` and `4` and peek at the rear element. Remove elements at the front and rear of the deque. Then ask if the deque is empty, and finally print the size of the deque.

```
int main() {
    ArrayDeque myDeque(5);

    myDeque.addFront(1);
    myDeque.addFront(2);
    std::cout << "Front element: " << myDeque.peekFront() <<
        std::endl;

    myDeque.addRear(3);
    myDeque.addRear(4);
    std::cout << "Rear element: " << myDeque.peekRear() <<
        std::endl;

    std::cout << "Removed from front: " << myDeque.removeFront()
        << std::endl;
    std::cout << "Removed from rear: " << myDeque.removeRear()
        << std::endl;

    std::cout << "Is deque empty? " << (myDeque.isEmpty() ?
        "true" : "false") << std::endl;
    std::cout << "Size of deque: " << myDeque.getSize() <<
        std::endl;

    return 0;
}
```

You should see the following output:

```
Front element: 2
Rear element: 4
Removed from front: 2
Removed from rear: 4
Is deque empty? false
Size of deque: 2
```

▼ Code

Your code should look something like this:

```
#include <iostream>
#include <stdexcept>

class ArrayDeque {
private:
    int capacity;
    int size;
    int front;
    int rear;
    int* dequeArray;

public:
    // Constructor
    ArrayDeque(int c) {
        capacity = c;
        size = 0;
```

```
        front = 0;
        rear = -1;
        dequeArray = new int[capacity];
    }

    // Destructor
    ~ArrayDeque() {
        delete[] dequeArray;
    }

    // Add element to the front
    void addFront(int value) {
        if (isFull()) {
            throw std::overflow_error("Deque is full");
        }

        front = (front - 1 + capacity) % capacity;
        dequeArray[front] = value;
        size++;
    }

    // Add element to the rear
    void addRear(int value) {
        if (isFull()) {
            throw std::overflow_error("Deque is full");
        }

        rear = (rear + 1) % capacity;
        dequeArray[rear] = value;
        size++;
    }

    // Remove element from the front
    int removeFront() {
        if (isEmpty()) {
            throw std::underflow_error("Deque is empty");
        }

        int temp = dequeArray[front];
        front = (front + 1) % capacity;
        size--;
        return temp;
    }

    // Remove element from the rear
    int removeRear() {
        if (isEmpty()) {
            throw std::underflow_error("Deque is empty");
        }

        int temp = dequeArray[rear];
        rear = (rear - 1 + capacity) % capacity;
        size--;
        return temp;
    }

    // Peek at the front element
```

```
    int peekFront() {
        if (isEmpty()) {
            throw std::underflow_error("Deque is empty");
        }

        return dequeArray[front];
    }

    // Peek at the rear element
    int peekRear() {
        if (isEmpty()) {
            throw std::underflow_error("Deque is empty");
        }

        return dequeArray[rear];
    }

    // Check if the deque is empty
    bool isEmpty() {
        return (size == 0);
    }

    bool isFull() {
        return (size == capacity);
    }

    int getSize() {
        return size;
    }
};

int main() {
    ArrayDeque myDeque(5);

    myDeque.addFront(1);
    myDeque.addFront(2);
    std::cout << "Front element: " << myDeque.peekFront() <<
        std::endl;

    myDeque.addRear(3);
    myDeque.addRear(4);
    std::cout << "Rear element: " << myDeque.peekRear() <<
        std::endl;

    std::cout << "Removed from front: " << myDeque.removeFront()
        << std::endl;
    std::cout << "Removed from rear: " << myDeque.removeRear()
        << std::endl;

    std::cout << "Is deque empty? " << (myDeque.isEmpty() ?
        "true" : "false") << std::endl;
    std::cout << "Size of deque: " << myDeque.getSize() <<
        std::endl;

    return 0;
}
```

Deque Using Doubly Linked List

Representing a Deque with a Doubly Linked List

Now let's discuss how to implement a deque using a doubly linked list. The advantages of using a doubly linked list over an array are that the doubly linked list can grow or shrink dynamically and it allows for efficient insertions and deletions at both ends.

Key Operations and Their Time Complexities

The efficiencies of a doubly linked list are most noticeable with adding to the front and rear of the deque. Here are the key operations for a doubly linked list-based deque and their respective time complexities:

- **addFront(element)**: $O(1)$
- **addRear(element)**: $O(1)$
- **removeFront()**: $O(1)$
- **removeRear()**: $O(1)$
- **peekFront()**: $O(1)$
- **peekRear()**: $O(1)$
- **isEmpty()**: $O(1)$
- **size()**: $O(1)$

Implementation

We first need to define the `Node` class, which has an attribute for the data stored as well as pointers for the previous and next nodes. In the constructor, set `data` to the parameter and set both pointers to `nullptr`.

```
class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
        this->prev = nullptr;
    }
};
```

Next, we need to make the `DequeLinkedList` class. The class has three attributes; `head` keeps track of the `head` node, `tail` keeps track of the `tail` node, and `size` keeps track of the `size` of the list. We can use C++'s default constructor to generate the default values of `nullptr` for both `head` and `tail` and `0` for `size`.

```
class DequeLinkedList {
private:
    Node* head;
    Node* tail;
    int size;

public:
    DequeLinkedList() {
        head = nullptr;
        tail = nullptr;
        size = 0;
    }

    void addFront(int data);
    void addRear(int data);
    int removeFront();
    int removeRear();
    int peekFront();
    int peekRear();
    bool isEmpty();
    int getSize();
};
```

To add an element to the front of the `deque`, create a new node with the given data. If the linked list is empty, set both `head` and `tail` to the new node. Otherwise, have the `next` pointer in the new node point to the old `head`. The `prev` pointer in the old head should point to the new node. Then, have `head` become the new node. Lastly, increment `size` by `1`.

```
void DequeLinkedList::addFront(int data) {
    Node* newNode = new Node(data);

    if (isEmpty()) {
        head = newNode;
        tail = newNode;
    } else {
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }

    size++;
}
```

Adding an element to the rear is a similar task. If the list is empty, both `head` and `tail` should point to the new node. Otherwise, the `prev` pointer of the new node should point to the old `tail`. The `next` pointer in the old `tail`

should point to the new node. Then have `tail` become the new node. Lastly, increment `size` by `1`.

```
void DequeLinkedList::addRear(int data) {
    Node* newNode = new Node(data);

    if (isEmpty()) {
        head = newNode;
        tail = newNode;
    } else {
        newNode->prev = tail;
        tail->next = newNode;
        tail = newNode;
    }

    size++;
}
```

Removing an element from the `front` of a `deque`, first check to see if the list is empty. If so, throw an exception with a message for the user. Then create a temporary node with the information of the old head. Have the `head` attribute become the `next` node in the list. If the new `head` is `nullptr`, set the `tail` to `nullptr` since the list is now empty. Otherwise, have `head->prev` become `nullptr`. Lastly, decrement `size` by `1` and return the data stored in the temporary variable.

```
int DequeLinkedList::removeFront() {
    if (isEmpty()) {
        throw std::underflow_error("Deque is empty");
    }

    Node* temp = head;
    head = head->next;
    if (head == nullptr) {
        tail = nullptr;
    } else {
        head->prev = nullptr;
    }
    int data = temp->data;
    delete temp;
    size--;
    return data;
}
```

Create the `removeRear` method to remove a node from the rear of the `deque`. Start by checking if the deque is empty. If so, throw an exception with a message to the user. Then create a temporary variable that has the information stored in the old tail. Set the new `tail` to the previous node in the list. If the new tail is `nullptr` set `head` to `nullptr` as well since the list is now empty. Otherwise, set `tail->next` to `nullptr`. Lastly, decrement `size` by `1` and return the data stored in the temporary variable.

```
int DequeLinkedList::removeRear() {
    if (isEmpty()) {
        throw std::underflow_error("Deque is empty");
    }

    Node* temp = tail;
    tail = tail->prev;
    if (tail == nullptr) {
        head = nullptr;
    } else {
        tail->next = nullptr;
    }
    int data = temp->data;
    delete temp;
    size--;
    return data;
}
```

To peek at the `front` and `rear` nodes, create the `peekFront` and `peekRear` methods. The `peekFront` method returns the data stored in the `head`, while the `peekRear` method returns the data stored in the `tail` node. Do not alter the `next` and `prev` pointers in these methods.

```
int DequeLinkedList::peekFront() {
    if (isEmpty()) {
        throw std::underflow_error("Deque is empty");
    }
    return head->data;
}

int DequeLinkedList::peekRear() {
    if (isEmpty()) {
        throw std::underflow_error("Deque is empty");
    }
    return tail->data;
}
```

Lastly, we have two helper methods. The `isEmpty` method returns if `head` is equal to `nullptr`, which describes an `empty` linked list. The `getSize` method is an accessor method that returns the `size` attribute.

```
bool DequeLinkedList::isEmpty() {
    return head == nullptr;
}

int DequeLinkedList::getSize() {
    return size;
}
```

Create the `main` method which instantiates a `doubly` linked list-based deque. Add the nodes `1` and `2`. Then peek at the front node. Add nodes `3` and `4` and peek at the rear node. Remove nodes at the front and rear of the deque.

Then ask if the deque is empty, and finally print the size of the deque.

```
int main() {
    DequeLinkedList myDeque;

    myDeque.addFront(1);
    myDeque.addFront(2);
    std::cout << "Front element: " << myDeque.peekFront() <<
        std::endl;

    myDeque.addRear(3);
    myDeque.addRear(4);
    std::cout << "Rear element: " << myDeque.peekRear() <<
        std::endl;

    std::cout << "Removed from front: " << myDeque.removeFront()
        << std::endl;
    std::cout << "Removed from rear: " << myDeque.removeRear()
        << std::endl;

    std::cout << "Is deque empty? " << (myDeque.isEmpty() ?
        "true" : "false") << std::endl;
    std::cout << "Size of deque: " << myDeque.getSize() <<
        std::endl;

    return 0;
}
```

You should see the following output:

```
Front element: 2
Rear element: 4
Removed from front: 2
Removed from rear: 4
Is deque empty? false
Size of deque: 2
```

▼ Code

Your code should look like this:

```
#include <iostream>
#include <stdexcept>

class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
        this->prev = nullptr;
    }
```

```
};

class DequeLinkedList {
private:
    Node* head;
    Node* tail;
    int size;

public:
    DequeLinkedList() {
        head = nullptr;
        tail = nullptr;
        size = 0;
    }

    void addFront(int data) {
        Node* newNode = new Node(data);

        if (isEmpty()) {
            head = newNode;
            tail = newNode;
        } else {
            newNode->next = head;
            head->prev = newNode;
            head = newNode;
        }

        size++;
    }

    void addRear(int data) {
        Node* newNode = new Node(data);

        if (isEmpty()) {
            head = newNode;
            tail = newNode;
        } else {
            newNode->prev = tail;
            tail->next = newNode;
            tail = newNode;
        }

        size++;
    }

    int removeFront() {
        if (isEmpty()) {
            throw std::underflow_error("Deque is empty");
        }

        Node* temp = head;
        head = head->next;
        if (head == nullptr) {
            tail = nullptr;
        } else {
            head->prev = nullptr;
        }
```

```
        int data = temp->data;
        delete temp;
        size--;
        return data;
    }

    int removeRear() {
        if (isEmpty()) {
            throw std::underflow_error("Deque is empty");
        }

        Node* temp = tail;
        tail = tail->prev;
        if (tail == nullptr) {
            head = nullptr;
        } else {
            tail->next = nullptr;
        }
        int data = temp->data;
        delete temp;
        size--;
        return data;
    }

    int peekFront() {
        if (isEmpty()) {
            throw std::underflow_error("Deque is empty");
        }
        return head->data;
    }

    int peekRear() {
        if (isEmpty()) {
            throw std::underflow_error("Deque is empty");
        }
        return tail->data;
    }

    bool isEmpty() {
        return head == nullptr;
    }

    int getSize() {
        return size;
    }
};

int main() {
    DequeLinkedList myDeque;

    myDeque.addFront(1);
    myDeque.addFront(2);
    std::cout << "Front element: " << myDeque.peekFront() <<
        std::endl;

    myDeque.addRear(3);
    myDeque.addRear(4);
```

```
    std::cout << "Rear element: " << myDeque.peekRear() <<
        std::endl;

    std::cout << "Removed from front: " << myDeque.removeFront()
        << std::endl;
    std::cout << "Removed from rear: " << myDeque.removeRear()
        << std::endl;

    std::cout << "Is deque empty? " << (myDeque.isEmpty() ?
        "true" : "false") << std::endl;
    std::cout << "Size of deque: " << myDeque.getSize() <<
        std::endl;

    return 0;
}
```

Analysis of Deque Implementations

Analyzing the Different Deque Implementations

We have seen how a deque can be implemented as either an array or a doubly linked list. The former gives you increased flexibility while the latter offers simplicity. Let's see how the two compare regarding time and space complexity.

Time Complexity Analysis

The table below summarizes the time complexity for common operations. Array-based deques could have a complexity of $O(n)$ in instances where the array needs to be resized. As long as the array stays the same size, these operations have a constant time complexity. Since a linked list is dynamic, there is no `isFull` method associated with this implementation.

Operation	Array-Based	Doubly Linked List-Based
Add to Front	$O(1)$ or $O(n)$	$O(1)$
Add to Rear	$O(1)$ or $O(n)$	$O(1)$
Remove from Front	$O(1)$	$O(1)$
Remove from Rear	$O(1)$	$O(1)$
Peek Front	$O(1)$	$O(1)$
Peek Rear	$O(1)$	$O(1)$
Is Empty	$O(1)$	$O(1)$
Is Full	$O(1)$	—

Space Complexity

Implementing a deque with either an array or a linked list does not change the space complexity. Both of these implementations will have as many elements or nodes as there is data to store. So the space complexity is linear in relation to the number of elements in the deque.

Array-Based	Doubly Linked List-Based
$O(n)$	$O(n)$

Advantages and Drawbacks

The time and space complexities are fairly similar between the two underlying data structures. That does not mean that they are interchangeable. There are a few important differences between array-based and linked list-based deques.

	Array-Based	**Doubly Linked List-Based**
Advantages	• All operations are done in constant time. • Does not require manipulating pointers.	• All operations are done in constant time. • The structure is dynamic and does not need to be resized.
Disadvantages	• The structure is static and requires additional overhead to resize.	• Uses extra memory for `prev` and `next` pointers for each node.

Formative Assessment 1

Formative Assessment 2